

Project of Computer Intelligence for Optimization

Group C25_Artificial_Selection

Artem Polikarpov (20250443)
Muhammad Khuzaima Bashir (20250441)
Muhammad Ammar (20250440)
Simon Sazonov (20221689)

Spring Semester 2025-2026

1. Abstract	2
2. Introduction	3
3. Representation and Fitness	3
3.1 Triangle Based Representation	3
3.2 Why This Representation Was Used	4
3.3 Rendering Procedure	4
3.4 Fitness Function	4
4. Genetic Algorithm Implementation	4
4.1 Selection	4
4.2 Elitism	4
4.3 Crossover	4
4.4 Mutation	5
5. Experimental Methodology	5
5.1 Diversity Diagnostics	5
5.2 One at a Time Experiments	6
5.3 Optuna Joint Tuning	6
5.4 Structural Improvements	6
5.5 Hybrid Approach and Long Run Convergence	7
6. Results	7
7. Challenge 2: Genetic Algorithm vs Simulated Annealing	7
8. Final Results	8
9. Limitations	8
10. Conclusion	9
References	9
Appendix A. Figures and Supporting Visual Results	10
Appendix A.1 Problem Setting and Output Examples	10
Appendix A.2 Baseline GA Diagnostics	10
Appendix A.3 One at a Time Hyperparameter Experiments	11
Appendix A.4 Main Performance Milestones	12
Appendix A.5 Visual Progression of Reconstructions	12
Appendix A.6 Challenge 2 Budget Design	13
Appendix A.7 Challenge 2 Results	13
Appendix A.8 Final Validation Results	14
Appendix A.9 Long Run Convergence	14
Appendix B. Configuration Summary	15
Appendix B.1 Baseline GA	15
Appendix B.2 Optuna Best	15
Appendix B.3 Final Best	15
Appendix B.4 SA Challenge Configuration	16
Appendix C. Code Structure	16

1. Abstract

This project approximates Vermeer's Girl with a Pearl Earring using 100 partially transparent triangles, each controlled by ten genes covering three vertices, an RGB colour, and an alpha channel. Candidates are rendered at 300×400 pixels using Porter-Duff compositing and scored pixel by pixel RMSE against the target. Because every triangle affects the appearance of all triangles drawn above it, the search space has no independent subproblems and resists simple greedy approaches.

We implemented a Genetic Algorithm as the primary optimiser, built around tournament selection, crossover at triangle level, multiple mutation operators, and elitism. Starting from a baseline without any tuning, we ran thirty independent seeds to establish a reliable reference mean RMSE of 28.40 ± 0.72 at 1,000 generations. One at a time parameter sweeps (30 seeds $\times$ 300 generations each as a short proxy budget, before full 1,000 generation revalidation) informed a joint Bayesian search using Optuna, which brought the mean down to 27.35 ± 0.57 . The most effective structural addition was a dynamic parameter schedule: starting with aggressive mutation and loose selection, then gradually tightening both, which pushed the mean to 26.43 ± 0.59 . Given more compute, a single extended run reached RMSE 20.3170 at generation 15,785.

For Challenge 2 we compared a previously tuned GA challenge configuration against Simulated Annealing on an equal evaluation budget of 63,000 fitness evaluations across thirty seeds. The headline means were almost identical: GA 25.12 ± 0.70 , SA 25.00 ± 4.41 . A paired Wilcoxon test ($p = 0.0606$) shows no significant difference at the 0.05 level. What distinguishes the algorithms is consistency: GA stayed within a narrow RMSE band across all thirty seeds, while SA showed much wider spread, with some seeds converging well below 25 and others stalling above 32. Which algorithm to prefer depends on whether consistently strong results matter more than the chance of an occasional excellent outcome.

2. Introduction

The core challenge is that triangles are composite in layer order: any gene change affects every layer drawn above it, making the search space not decomposable into independent parts and motivating the population based approach.

3. Representation and Fitness

3.1 Triangle Based Representation

Each candidate solution is a flat list of 1,000 integer genes. The list is divided into 100 blocks of 10 genes, where each block represents one triangle (100 blocks $\times$ 10 genes = 1,000 genes total).

Gene group	Meaning	Range
x1, y1, x2, y2, x3, y3	three triangle vertices	x in [0, 300], y in [0, 400]
R, G, B	triangle color	[0, 255]
A	alpha transparency	initialized in [20, 180], later clamped to [0, 255]

Position in the list matters. Triangle 0 is drawn first onto a blank canvas; triangle 99 is drawn last. This z ordering is baked into the genome and remains fixed throughout evolution.

3.2 Why This Representation Was Used

We settled on integer triangle gene blocks because a triangle is a coherent visual unit. Its vertex coordinates, colour channels, and alpha value only make sense together. Splitting at bit level would corrupt vertex coordinates across parents, producing incoherent shapes. Binary encoding was rejected for this reason; pixel level encoding is excluded by the specification; opaque triangles cannot approximate smooth colour gradients; and fixed z ordering was retained after z order swap showed no consistent improvement in the ablation phase.

3.3 Rendering Procedure

Each triangle is rendered in genome order onto a 300 by 400 RGBA canvas using Porter-Duff compositing; the result is compared pixel by pixel to the target image.

3.4 Fitness Function

Fitness is RMSE between the rendered image and the target, as specified in the brief. RMSE penalises larger pixel level errors more than small ones and is directly interpretable as average channel deviation, which makes it easy to reason about improvements.

4. Genetic Algorithm Implementation

The core loop follows Holland (1975). Our implementation uses tournament selection with elitism, triangle level uniform crossover, and a family of mutation operators, each described in the sections below.

The baseline configuration is listed in Appendix B.1.

4.1 Selection

We used tournament selection throughout. The tournament size k controls selection pressure directly: larger k favours the current best more strongly, smaller k gives weaker individuals a better chance of being selected.

4.2 Elitism

Each generation, the best one (baseline) or four (tuned) individuals carry over unchanged before crossover and mutation fill the rest. This prevents a good solution found at generation

400 from being accidentally destroyed by the next crossover, which is a real risk when the population is only 30 to 63 individuals.

4.3 Crossover

Crossover is implemented at triangle boundaries. Uniform triangle crossover, where each of the 100 triangle slots is drawn independently from one parent or the other, proved clearly better than single point crossover in the OAT experiments ($\Delta\text{RMSE} = 3.35$, $p < 0.05$). Single point crossover passes long consecutive blocks from one parent, preserving local z ordering but sacrificing the broader mixing effect that helps on this problem.

4.4 Mutation

The mutation module includes vertex shift, colour shift, alpha shift, triangle replace, triangle swap, and geometric perturbation.

All mutation operators produced identical means at 300 generations. Optuna resolved the tie at full budget, selecting `triangle_replace`, the most disruptive option. The dynamic schedule explains this: high early mutation probability keeps diversity alive through the collapse phase, and the scheduled rate reduction later lets the population refine rather than thrash. The two mechanisms are complementary.

5. Experimental Methodology

Rather than guessing a good configuration and committing to it, we worked through a staged process where each step answered one specific question before we moved on. The table below maps out that chain. Throughout all experiments, paired Wilcoxon signed rank tests are used when comparing matched seed results and Mann-Whitney U tests for independent samples; multiple comparisons are Holm corrected.

Stage	Question answered
Baseline GA	Does the representation and GA work?
Diversity diagnostics	Does the baseline converge prematurely?
One at a time tuning	Which parameter ranges are promising?
Optuna tuning	Which parameter combination performs best jointly?
Structural ablation	Do new operators improve over tuning alone?
Challenge 2	How does GA compare with SA under a fair budget?

Multi-seed validation	Is the final GA stable across seeds?
Long convergence run	How much further can the representation improve given a larger compute budget?

5.1 Diversity Diagnostics

The baseline GA logged genome entropy, mean pairwise distance, and fitness variance alongside best RMSE. Figure A.2 shows the result: population diversity collapsed within the first thirty generations while fitness kept improving all the way to generation 1,000. The GA was exploiting a narrow region for 97% of its budget, which informed every subsequent structural decision.

5.2 One at a Time Experiments

Nine one at a time experiments each varied a single GA parameter across a small grid while fixing everything else at baseline values: population size, elite size, mutation rate, crossover type, mutation type, tournament size, crossover probability, combined mutation/crossover regimes, and mutation step sizes. Each grid point ran 30 seeds for 300 generations, giving enough statistical power to separate genuinely different values from noise.

OAT established data driven search bounds for Optuna rather than final hyperparameters directly. The clearest result was uniform crossover beating single point by 3.35 RMSE ($p < 0.05$). Larger mutation steps (vertex=50, colour=80, alpha=50) outperformed defaults by 1.22 RMSE ($p < 0.05$). Population size and tournament size showed monotone gradients favouring their upper tested values, extending both bounds upward. Mutation type produced a flat result, with all operators identical at 300 generations, leaving Optuna to resolve the tie at full budget. The full OAT results are in Figure A.3.

5.3 Optuna Joint Tuning

With OAT bounds established, Optuna searched jointly across all parameters. Each of 50 trials was first evaluated with a single seed at 300 generations; the top three were revalidated at 30 seeds $\times$ 1,000 generations. The TPE sampler concentrates new trials in promising regions, making 50 trials far more informative than a coarse grid.

The winning configuration is listed in Appendix B.2. Key parameters: population 63, tournament size 7, elite 4, triangle_replace mutation with vertex_step=58, colour_step=91, alpha_step=50.

5.4 Structural Improvements

With hyperparameters fixed at the Optuna configuration available at that stage of development, we tested five structural additions one at a time: geometric mutation, triangle innovation replace, z order swap, fitness sharing, and a dynamic parameter schedule. Each ran for 300 generations against the same 30 paired seeds. None improved significantly at that short budget, but two had confidence intervals whose lower bounds dipped below zero, so they qualified for a full 1,000 generation re-run under the pre-registered promotion rule.

At 1,000 generations, the dynamic schedule won clearly. It reduces mutation probability linearly from 0.15 to 0.03 and tightens tournament size from 2 to 5 over the run, reaching 26.43 ± 0.59 against Optuna's 27.35 ± 0.57 . A paired Wilcoxon test confirmed the gain: $\Delta\text{RMSE} = -0.92$ [95% CI $-1.18, -0.65$], $p < 0.0001$. The logic is that high early mutation keeps the population diverse through the collapse phase, and the scheduled tightening later converts that diversity into refined exploitation. Fitness sharing, which we expected to help most, produced no measurable improvement ($\Delta\text{RMSE} = +0.05$, $p = 0.72$). The penalty fires uniformly across an already collapsed population, which leaves tournament selection rankings unchanged and gives the mechanism nothing to work with. Z order swap also showed no consistent benefit.

5.5 Hybrid Approach and Long Run Convergence

As a supplementary test, 6,000 SA evaluations were applied to each of the 30 final GA solutions. The hybrid reached 26.43 ± 0.59 against the GA only result of 26.43 ± 0.59 ($\Delta\text{RMSE} = -0.0016$, $p = 0.0022$), statistically significant but negligible in practice, confirming the dynamic schedule had already driven solutions to a local optimum. The same configuration ran to a ceiling of 20,000 generations with early stopping halted at generation 15,785, reaching RMSE 20.3170, a single unreplicated observation included as a ceiling estimate (Figure A.9).

6. Results

The staged optimization produced clear milestones. Starting from a baseline of 28.40 ± 0.72 RMSE, OAT established that uniform crossover outperforms single point by 3.35 RMSE ($p < 0.05$) and larger mutation steps outperform defaults by 1.22 RMSE ($p < 0.05$). Optuna joint search over all OAT derived bounds gave 27.35 ± 0.57 (gain: 1.05 RMSE, 3.7%). The dynamic parameter schedule was the decisive structural addition: reaching 26.43 ± 0.59 , a further gain of 0.92 RMSE over Optuna ($\Delta\text{RMSE} = -0.92$ [95% CI $-1.18, -0.65$], $p < 0.0001$). SA polishing after GA convergence added a statistically significant but negligible 0.0016 RMSE ($p = 0.0022$). Given extended compute, the same configuration reached 20.32 RMSE at generation 15,785. Full milestone detail appears in Figure A.4.

7. Challenge 2: Genetic Algorithm vs Simulated Annealing

For Challenge 2, we compared Simulated Annealing against a previously tuned GA challenge configuration under the same evaluation budget of 63,000 fitness evaluations. Both algorithms received exactly 63,000 evaluations: the SA outer cycle count (150 inner iterations over 420 outer cycles) was derived from the GA population size. The GA result in Challenge 2 should be interpreted as an equal-budget GA comparator run, not as the final submitted FINAL_BEST_CONFIG validation. The final submitted GA configuration is reported separately in Section 8.

Algorithm	Evaluation budget
Final GA	63,000 evaluations

Table 8 summarises the 30 seed results for both algorithms. The per seed distributions are shown in Figure A.7.

Algorithm	Mean RMSE	Sample SD	Success rate
Genetic Algorithm	25.12	0.70	73.3%
Simulated Annealing	25.00	4.41	83.3%

Table 8. Thirty-seed GA versus SA comparison under equal 63,000 evaluation budget. Before running either algorithm we registered a success threshold of RMSE 25.56 (90% of the baseline mean of 28.40). Against that, GA succeeded in 22 of 30 seeds (73.3%) and SA in 25 of 30 (83.3%). SA had the higher success rate.

Because seeds are matched (the same seed number initialises both algorithms identically), a paired Wilcoxon signed rank test is the appropriate choice. That test gave a statistic of 141.0 and $p = 0.0606$, with a mean paired difference of $\Delta\text{RMSE} = +0.12$ [95% CI $-1.52, +1.54$]. The confidence interval straddles zero comfortably, and the p-value does not cross the $\alpha = 0.05$ threshold. On this evidence, the two algorithms perform comparably under the matched budget.

8. Final Results

The dynamic schedule configuration from Section 5.4 is our final submitted answer. The question this section asks is straightforward: does it hold on seeds the experiments never saw? We re-ran on thirty independent validation indices, displayed as 0–29 in the notebook but internally offset to random seeds 100–129, keeping the validation pool distinct from all prior tuning and comparison experiments. The mean came out at 26.18 ± 0.84 , consistent with the 26.43 ± 0.59 from Section 5.4 and well within normal seed variability. The best seed reached 24.18 and the worst 27.66; every seed beat the baseline mean of 28.40. Four of thirty seeds (13.3%) fell below the pre-registered success threshold of 25.56, compared to zero in the baseline pool. The result confirms the dynamic schedule generalises and was not fit to a particular set of random initialisations.

A Mann-Whitney U test comparing the validation seeds against the baseline seeds gave $U = 882.0$ and $p = 1.78 \times 10^{-10}$, with a mean improvement of $\Delta\text{RMSE} = 2.21$ [95% CI 1.83, 2.60]. The improvement is large and unambiguous. This is the number we would defend: 26.18 RMSE on 30 independent seeds, with every seed outperforming the untrained baseline. The full per seed distribution appears in Figure A.8.

9. Limitations

Three limitations shaped what we could conclude. Computing was the binding constraint: each full 30 seed experiment takes several hours even with parallelisation, which limited how many grid points we could explore and how many seeds we could use per trial.

RMSE weights every pixel equally, which undervalues fine portrait details such as the pearl sheen and the catch light, which occupy fewer pixels than the background. A perceptual metric such as SSIM would address this, but switching metrics would redefine the problem.

Both algorithms are stochastic, and 30 seeds represent one draw from a high variance process. The p-value (0.0606) sits just above $\alpha = 0.05$ and could shift with a different seed draw. We report the outcome as-is.

10. Conclusion

The key design decision was keeping each triangle as an intact ten gene block. This gives crossover a meaningful unit to recombine; splitting vertex coordinates across parents would produce incoherent shapes. The tournament selection parameters, the mutation operators, and the dynamic schedule all followed from the experimental findings described above.

Fitness sharing was expected to be the key intervention. Diversity collapse by generation 30 seemed to call for it directly, but it produced no measurable improvement. The penalty fires uniformly across an already collapsed population, leaving selection rankings unchanged. The dynamic schedule was the decisive change instead: invisible at 300 generations but worth nearly a full RMSE unit by generation 1,000. Triangle_replace was also surprising. It was indistinguishable from other operators at short budget but won clearly at full budget, because the scheduled rate reduction lets the population refine what the aggressive early phase opened up.

The GA versus SA comparison showed that algorithm choice depends on what you optimize for. Under an equal 63,000 evaluation budget using a previously tuned GA challenge configuration, the mean RMSE difference was not statistically significant ($p = 0.0606$). SA had the higher success rate (83.3% vs 73.3%) but six times the standard deviation, with some seeds converging well and others stalling, whereas GA delivered consistently strong results every time. Whether that consistency or the chance of a better outcome matters more is a practical question, not a statistical one. The long run result (20.3170 RMSE at generation 15,785) shows the representation has headroom well beyond the 1,000 generation budget.

The most promising future direction is incremental triangle rendering: optimising one triangle at a time before introducing the next gives the GA a natural curriculum and avoids wasted computation on covered layers. A perceptual fitness function such as SSIM would better target fine details that RMSE underweights.

References

Akiba, T., Sano, S., Yanase, T., Ohta, T., & Koyama, M. (2019). Optuna: A next-generation hyperparameter optimization framework. Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining.

Holland, J. H. (1975). Adaptation in Natural and Artificial Systems. University of Michigan Press.

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. Science, 220(4598), 671-680.

Appendix A. Figures and Supporting Visual Results

All figures referenced in the main report are collected here. Where the main text gives an interpretation, the appendix gives the full visual evidence.

Appendix A.1 Problem Setting and Output Examples

Figure A.1 places the target image alongside the baseline GA output and the long run output, giving an immediate sense of how much visual quality improves with tuning and compute.

Figure A.1 — Problem Setting and Output Examples



Figure A.1. Target image, baseline reconstruction, and long run output.

Appendix A.2 Baseline GA Diagnostics

The diversity collapse described in Section 5.1 is visible in Figure A.2: mean pairwise genome distance drops sharply in the first thirty generations and stays flat for the rest of the run, while best RMSE continues falling. This separation is what motivated the structural experiments in Section 6.

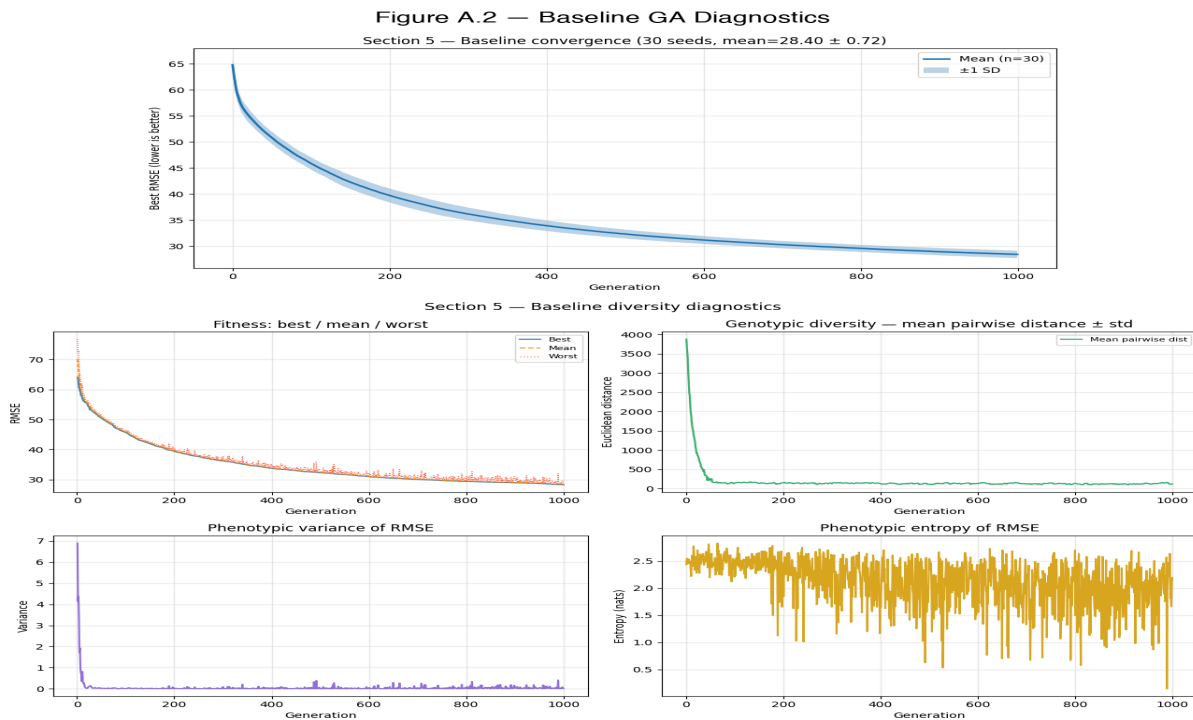


Figure A.2. Baseline convergence, diversity, fitness variance, and diversity and fitness relationship.

Appendix A.3 One at a Time Hyperparameter Experiments

Figure A.3 summarises all nine one at a time experiments. Each panel shows the 30 seed RMSE distribution for each tested value of one parameter, with the baseline value highlighted and equivalence groups marked. These charts determined the Optuna search space used in Section 5.3.

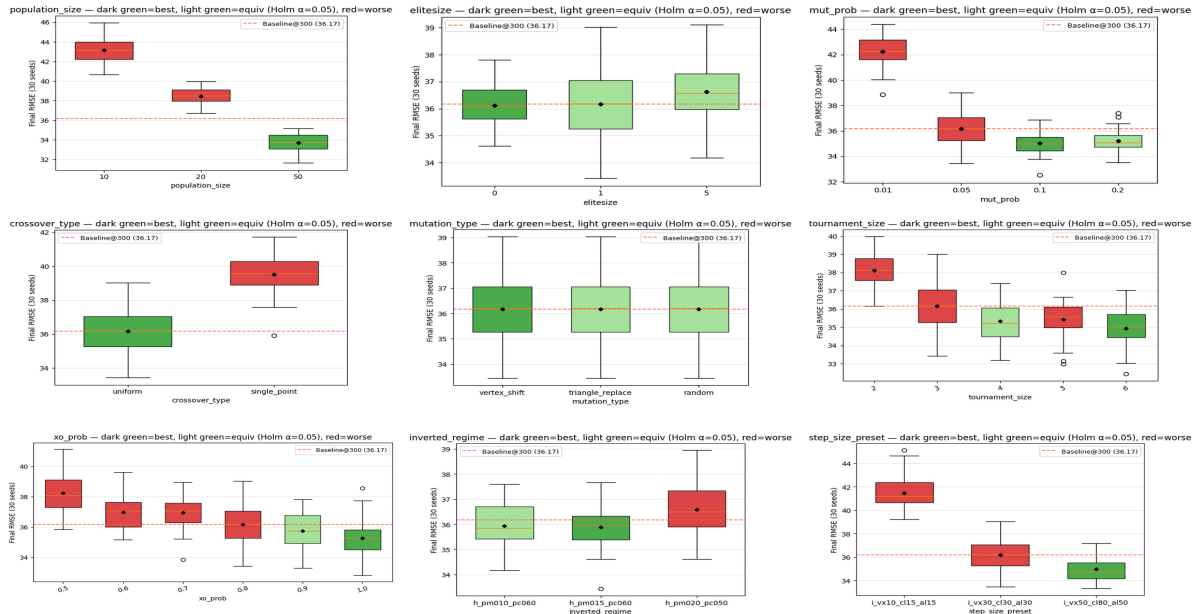


Figure A.3. One at a time experiment results for major GA parameters.

Appendix A.4 Main Performance Milestones

Figure A.4 places the four main RMSE milestones(baseline, Optuna tuned, dynamic schedule, long run)on a single bar chart, making the relative size of each improvement visible at a glance. Referenced in Section 5.4.

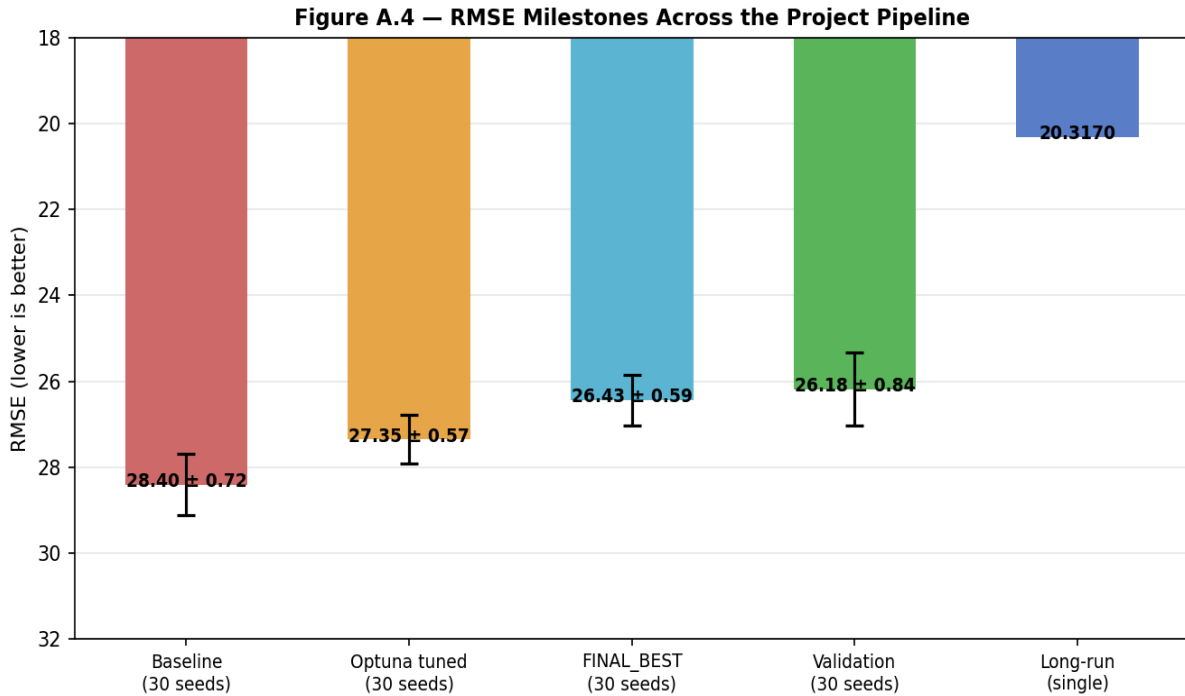


Figure A.4. Main RMSE milestones across the project pipeline (baseline → Optuna → dynamic schedule → long run).

Appendix A.5 Visual Progression of Reconstructions

Numbers alone do not fully capture what an improvement from 28.40 to 26.43 RMSE looks like. Figure A.5 shows the actual rendered outputs at each stage side by side, giving the visual context for the numerical progression discussed in Section 5.4.

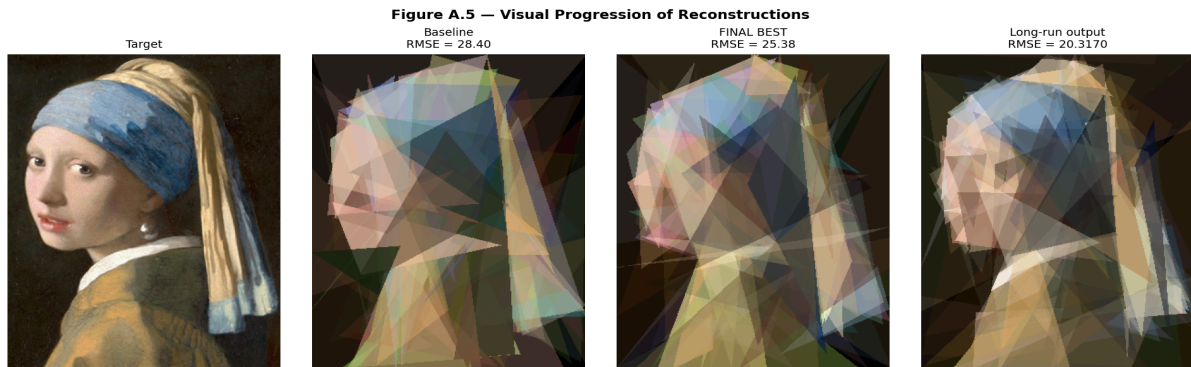


Figure A.5. Visual progression from target and baseline to tuned and long run reconstructions.

Appendix A.6 Challenge 2 Budget Design

Figure A.6 shows the equal budget design described in Section 7: both GA and SA receive exactly 63,000 fitness evaluations, with the SA outer cycle count derived directly from that constraint.

Figure A.6 — Equal Evaluation Budget Design (GA vs SA)

Algorithm	Population	Generations	Evals/Gen	Total Evaluations
Genetic Algorithm	63	1000	63	63,000
Simulated Annealing	1 (single)	420 outer cycles	150 inner steps	63,000

Figure A.6. Equal evaluation budget design for GA and SA.

Appendix A.7 Challenge 2 Results

Figure A.7 shows the full 30 seed results for both algorithms. The wide spread in SA's distribution, with some seeds converging below RMSE 23 while others stalled above 32, contrasts with GA's tighter clustering between 24 and 27.

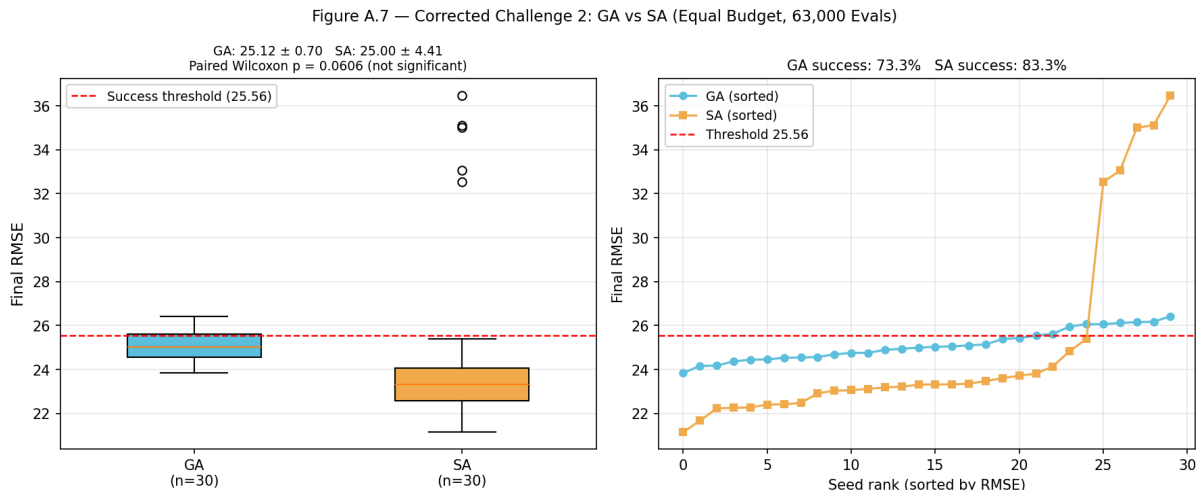


Figure A.7. GA vs SA comparison across thirty seeds.

Appendix A.8 Final Validation Results

Figure A.8 shows the thirty fresh validation seeds from Section 8, plotted alongside the baseline distribution for comparison. Every validation seed falls below the baseline worst; the distributions do not overlap.

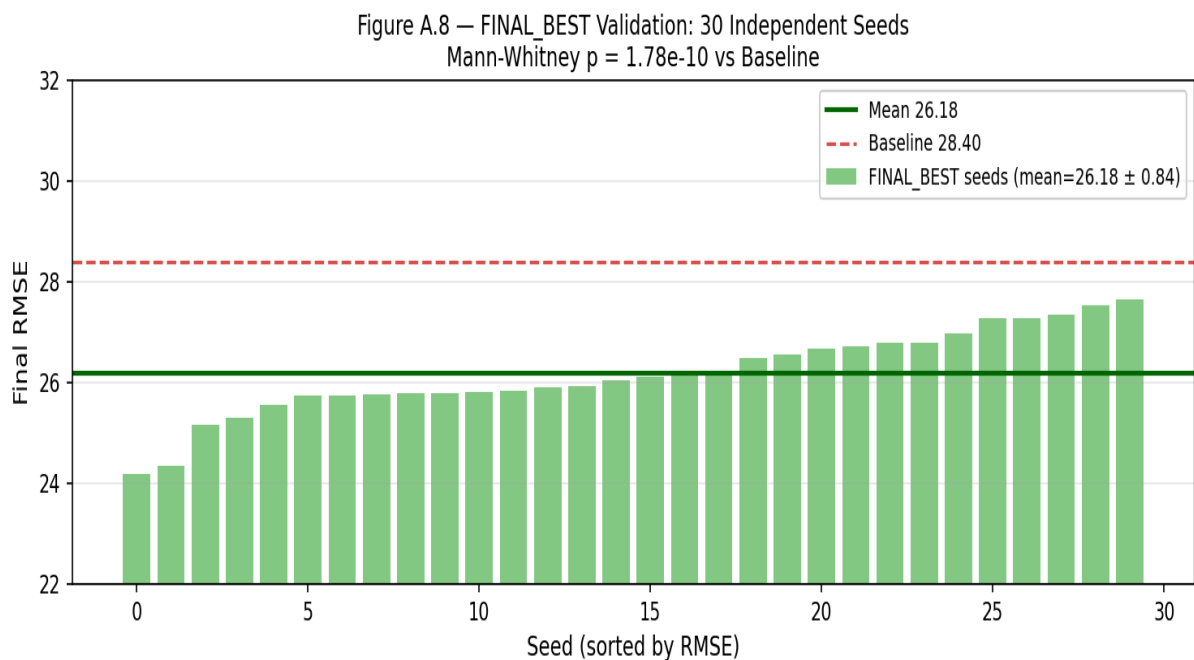


Figure A.8. Validation of the final GA configuration across thirty independent seeds.

Appendix A.9 Long Run Convergence

Figure A.9 shows the convergence curve for the long run described in Section 5.5. The early rapid descent, the gradual flattening after generation 5000, and the final stop at generation 15,785 are all visible.

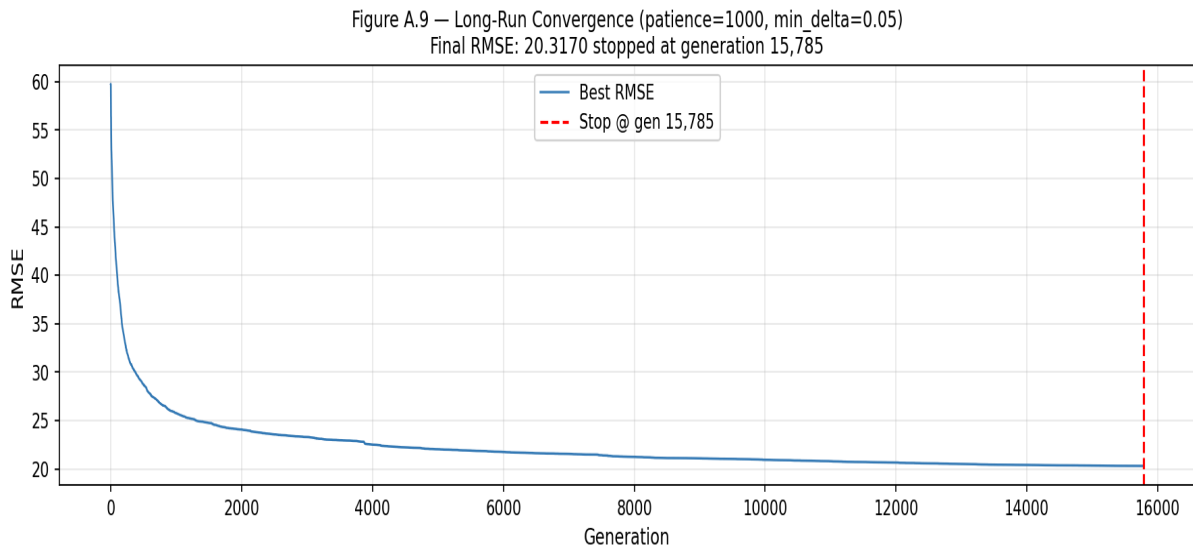


Figure A.9. Long run convergence with early stopping.

Appendix B. Configuration Summary

Appendix B.1 Baseline GA

Parameter	Value
population size	30
generations	1000
tournament size	3
elite size	1
crossover probability	0.8
mutation probability	0.05
crossover type	uniform
mutation type	random

Appendix B.2 Optuna Best

Parameter	Value
-----------	-------

population size	63
generations	1000
tournament size	7
elite size	4
crossover probability	1.0
mutation probability	0.1016
crossover type	uniform
mutation type	triangle replace
vertex step	58
color step	91
alpha step	50

Appendix B.3 Final Best

The final configuration starts from OPTUNA_BEST_CONFIG and adds three dynamic schedule parameters, shown below:

Parameter	Value
dynamic schedule	True
mutation probability start → end	0.15 → 0.03
tournament size start → end	2 → 5

Appendix B.4 SA Challenge Configuration

Parameter	Value
initial temperature C	20.0
inner loop length L	150
cooling factor H	1.05
outer cycles	420
total evaluations	63,000

Appendix C. Code Structure

Topic	File
Representation and rendering	library/problems/triangle_image.py
Abstract solution interface	library/problems/solution.py
GA loop and logging	library/algorithms/geneticalgorithms/ga.py
Selection operators	library/algorithms/geneticalgorithms/selection.py
Crossover operators	library/algorithms/geneticalgorithms/crossover.py
Mutation operators	library/algorithms/geneticalgorithms/mutation.py
Diversity metrics	library/algorithms/geneticalgorithms/diversity.py
Simulated Annealing	library/algorithms/simulated_annealing.py
Parallel seed execution	utils.py
Notebook orchestration	main_notebook.ipynb